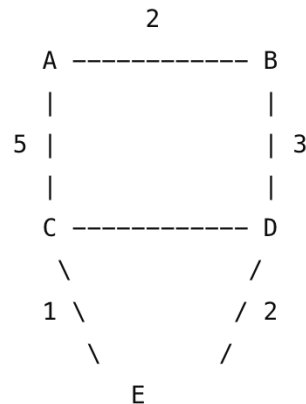


Exercise 2: Link State and Distance Vector Routing

Name: พิสิษฐ วงศ์ถวิล Student ID: 6710405125 Date: 21/9/2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

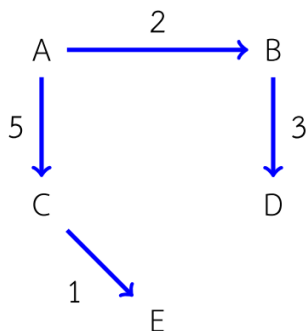
Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	Inf, -	Inf, -
1	B	2, A	5, A	5, B	Inf, -
2	C	2, A	5, A	5, B	6, C
3	D	2, A	5, A	5, B	6, C
4	E	2, A	5, A	5, B	6, C

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



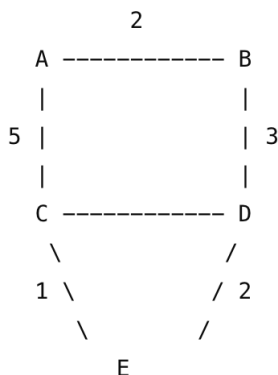
Destination	Shortest Path	Total Cost
B	A -> B	2
C	A -> C	5
D	A -> B -> C	5
E	A -> C -> E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A -> B
C	5	C	A -> C
D	5	B	A -> B -> D
E	6	C	A -> C -> E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2 + 0 = 2$	$5 + 4 = 9$	2	B
C	$2 + 4 = 6$	$5 + 0 = 5$	5	C
D	$2 + 3 = 5$	$5 + 1 = 6$	5	B
E	$2 + 5 = 7$	$5 + 1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A -> B
C	C	5	A -> C
D	B	5	A -> B -> C
E	C	6	A -> C -> E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

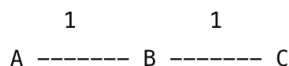
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
Routers C and D detect the C-D link failure.	Detect: Routers C and D detect the failure and update their own distance vector tables (setting the distance to D/C as infinity).
Flood: C and D flood Link State Packets (LSPs) immediately to inform all routers in the network about the link failure.	Propagate: C and D send their updated distance vector tables only to their directly connected neighbors.
Update: All routers receive the LSPs and update their Link State Database (LSDB) by removing the C-D link.	Incorrect Info: Routers may temporarily have incorrect routing information (potentially causing routing loops) because they rely on stale routes from neighbors instead of having a global view of the topology.
Calculate: Every router independently runs Dijkstra's algorithm to compute the new Shortest Path First (SPF) tree and routes.	Converge: The network eventually converges after multiple rounds of DV exchanges until all routing tables stabilize with the correct costs.

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

Because router B detects the link to C has failed, but it receives stale information from router A stating "A can reach C at a cost of 2". Since B does not know that A's path to C goes through B itself, B updates its next hop for C to point back to A. Meanwhile, A still points to B, creating a continuous routing loop.

2. Show how the estimated distance to C can increase over successive iterations.

Round 1: B adds 1 to A's advertised cost ($2+1 = 3$) and sends this update to A. Round 2: A adds 1 to B's updated cost ($3+1 = 4$) and sends it back to B. Round 3: B adds 1 to A's cost ($4+1 = 5$). The cost will continue to increase back and forth over successive iterations.

3. What is count-to-infinity?

It is a routing problem where the estimated distance (cost) to an unreachable destination continuously increases to infinity before the routers finally realize the destination is completely unreachable, causing extremely slow convergence.

4. Why does it occur in Distance Vector routing?

Because DV relies on "routing by rumor." Routers do not possess a complete network topology map, so they do not know if the path advertised by a neighbor actually routes back through themselves.

5. Name a mitigation approach.

Split Horizon, Poison Reverse, Route Poisoning, or Hold-down timers.

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Dijkstra's Algorithm	Bellman-Ford Algorithm
Information maintained	Full network topology database (LSDB)	Distance table of neighboring routers
Information exchanged	Link state of its own interfaces (LSPs) to everyone	Own entire distance vector table to neighbors only
Algorithm	Global (computes paths independently)	Decentralized (relies on neighbors' computations)
Topology knowledge	Full Topology (global view)	Partial / Neighbors Only (no global map)
Routing updates	Event-driven (only when topology changes)	Periodic intervals or when changes occur
Convergence	Fast	Slow
Routing loops	Extremely rare (except briefly during updates)	Common during the convergence process
Count-to-infinity	Does not occur	Prone to occur (requires mitigation techniques)
Example protocol	OSPF, IS-IS	RIP, EIGRP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

Chosen approach: Link State Routing (e.g., OSPF)

Advantages (Justification): **1. Fast Convergence:** When a link fails, LS immediately floods the topology changes to all routers. This fulfills the enterprise requirement for low downtime and fast convergence. **2. Accurate Topology Knowledge:** Every router maintains an identical, complete network map (LSDB). This prevents routing loops and the count-to-infinity problem, ensuring

highly efficient routing. **3. Hierarchical Design Support:** For a 500-router network, OSPF allows dividing the network into smaller areas (e.g., Backbone Area 0). This significantly reduces the LSDB size and CPU processing load on individual routers.

Disadvantages: - Consumes higher computational resources (CPU and RAM) because routers must continuously run Dijkstra's algorithm and store the full LSDB. - Configuration and maintenance are much more complex, requiring highly skilled network administrators compared to DV routing.

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

Algorithmic Analysis Comparison: - **Information Maintained:** LS maintains data for all links in the network (complexity $O(E)$). DV only maintains destination tables (complexity $O(N)$). - **Communication Overhead:** LS uses flooding for changes, generating $O(N \times E)$ overhead. DV sends its entire table to neighbors, generating $O(N)$ overhead per exchange. - **Computation:** LS uses Dijkstra, requiring $O(N \log N + E)$ computation per router (high CPU usage). DV uses the decentralized Bellman-Ford equation, requiring less computation per device. - **Convergence & Scalability:** LS converges in parallel (each router computes its own path instantly), making it fast. DV converges sequentially (hop-by-hop), making it slow and vulnerable to routing loops.

Why large-scale intra-domain routing favors Link State: Although LS has a higher initial CPU and bandwidth overhead, modern hardware and hierarchical routing features (like OSPF Areas) easily resolve these issues. In large-scale networks, "slow convergence" and "routing loops" are completely unacceptable. Link State protocols eliminate these critical issues, making them the superior and most stable choice for massive enterprise scalability.